Графовые алгоритмы на линейной алгебре: LAGraph против Spla

Блок 1 · BFS, SSSP, PageRank, Triangle counting

Дмитриевцев Алексей Сергеевич, группа 23.Б-11

03.11.2026

Окружение и методика

Стенд:

- CPU: 12th Gen Intel Core i7-12650H (6 P- и 4 E-ядра, 16 потоков), ОЗУ 16 ГБ.
- GPU: NVIDIA GeForce RTX 3050 Laptop GPU (OpenCL, драйвер NVIDIA).
- Windows 11, MSVC 2022; SuiteSparse:GraphBLAS 10.5.1 + LAGraph 1.2.2 — 1 и 16 потоков.
- Spla (коммит `b81d1e2`): примеры `examples/*.cpp` на GPU; для сравнения — её CPU-бэкенд.

Методика

- Граф неориентированный, без петель; SSSP — веса 1.
- 1 прогрев + 10 запусков; BFS/SSSP — из 3 общих стартовых вершин; столбец — медиана, усы — 25–75 %.
- Разброс внутри серии: межквартильный размах — 2–6 % медианы.
- Корректность: число уровней BFS и достигнутых вершин сверено с независимым расчётом (scipy).

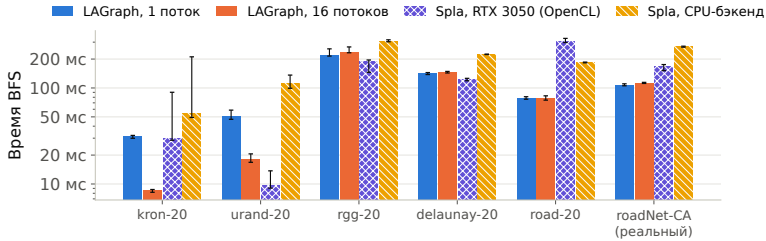
Графы: 5 семейств × 3 размера и roadNet-CA

Семейство	Тип	$\lvert V\rvert = 2^k$	$\lvert E\rvert$	ср. степень	макс. степень	уровней BFS
kron_g500-logn*k*	соцсеть	2^{16}, 2^{18}, 2^{20}	909.2K–15.7M	27.8–29.9	9.7K–65.0K	6–7
urand_n*k*	случайный	2^{16}, 2^{18}, 2^{20}	524.2K–8.4M	16.0	35–38	7–8
rgg_n_2_*k*_s0	геометрический	2^{16}, 2^{18}, 2^{20}	343.7K–6.9M	10.5–13.2	27–34	150–799
delaunay_n*k*	триангуляция	2^{16}, 2^{18}, 2^{20}	196.6K–3.1M	6.0	18–20	104–440
road_n*k*	дороги	2^{16}, 2^{18}, 2^{20}	83.7K–1.3M	2.5–2.6	5	295–1593

Синтетика: число вершин — параметр генератора, $2^{16} \to 2^{18} \to 2^{20}$ (шаг ×4), одинаковое у всех семейств; число рёбер задаёт средняя степень семейства. Генераторы — как у SuiteSparse (DIMACS10, Graph500); «дороги» — граф относительного соседства. roadNet-CA из набора Spla: 2 млн вершин, 2.8 млн рёбер.

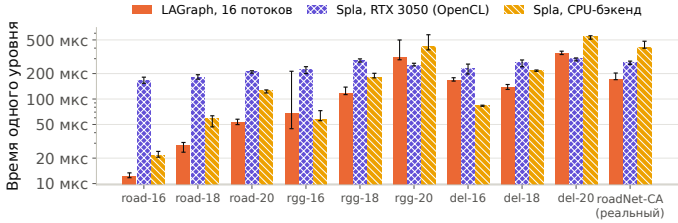
Главное различие графов: 6–8 уровней BFS против сотен и тысяч.

BFS: результат определяет ширина уровня



- urand-20: 8 уровней, до 705 тыс. вершин на уровне — накладные теряются в работе, GPU выигрывает.
- road-20: 1436 уровней по ≈730 вершин — работы нет, Spla платит только накладные. roadNet-CA (≈3 тыс. на уровень) — проигрыш меньше; rgg, delaunay (1.4–2.6 тыс.) — паритет.
- kron-20, длинный ус (вершина 177991): на 2-м уровне 5143 хаба — < 5 % вершин, и Spla делает push; LAGraph считает рёбра фронта (правило Бимера) и делает pull.
- 16 потоков ускоряют LAGraph только на kron и urand (в 3.6 и 2.8 раза): потокам нужен широкий уровень.

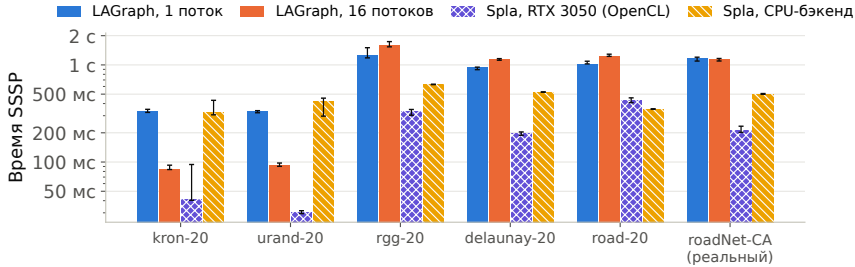
BFS: цена одного уровня



- У Spla на GPU уровень стоит 170–300 мкс и почти не растёт с графом (road-16 → road-20: 166 → 210 мкс при росте в 16 раз): основное время — не работа, а 5–8 запусков ядер и ожидание счётчика фронта на хосте.
- Проверка: тот же алгоритм на CPU-бэкенде Spla (без OpenCL, 1 поток) на road тратит 21–127 мкс на уровень; по профилю Spla (облако, POCL) сами ядра — 12–20 % времени операций, остальное — очередь и хост.
- У LAGraph уровень — несколько вызовов GraphBLAS в памяти CPU: на road 12–53 мкс, растёт вместе с числом вершин на уровне. На rgg/delaunay 2^{20} (1.4–2.6 тыс. вершин) работа LAGraph доходит до «пола» Spla — паритет.

Время уровня: для каждого запуска BFS его время делится на число уровней из этой стартовой вершины; на графике — медиана по 30 запускам (10 × 3 вершины). Отдельно уровни не замерялись.

SSSP: Spla на GPU быстрее в 2–5 раз



- Обе считают SSSP шагами по фронту, как BFS. Но цена перехода от BFS к SSSP разная: у Spla SSSP дороже её BFS в 1.3–3.1 раза, у LAGraph — в 5–13 раз.
- Spla — Bellman–Ford: то же умножение вектора на матрицу, что в BFS, только в полукольце $(\min, +)$.
- LAGraph — delta-stepping: на каждую «корзину» расстояний — цикл из нескольких операций GraphBLAS (выбрать корзину, релаксировать, слить) и нет перехода на pull, который ускоряет его BFS.
- Поэтому на road-20 Spla выигрывает меньше всего (2.4×): там и у Spla 1436 дорогих шагов. Параметр Δ — в приложении.

PageRank: ошибка в примере Spla

Как должно быть

$$p_i = \frac{1-\alpha}{N} + \alpha \sum_{j \in N(i)} \frac{p_j}{\deg(j)}, \qquad A_{ij} = \frac{\alpha}{\deg(j)}$$

Знаменатель — степень вершины, *отдающей* ранг.

Что в `examples/pr.cpp`, строка 87

```
A->set_float(Ai[k], Aj[k],
             alpha / A_degrees[Ai[k]]);
```

$A_{ij} = \alpha / \deg(i)$ — степень вершины, *получающей* ранг.

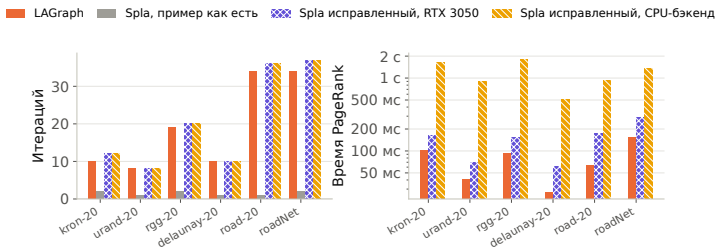
Следствие: при $p^{(0)} = 1/N$

$$p_i^{(1)} = \frac{1-\alpha}{N} + \deg(i) \cdot \frac{\alpha}{\deg(i)} \cdot \frac{1}{N} = \frac{1}{N}$$

$1/N$ — неподвижная точка: невязка 0, стоп после 1 итерации (2 — при изолированных вершинах).

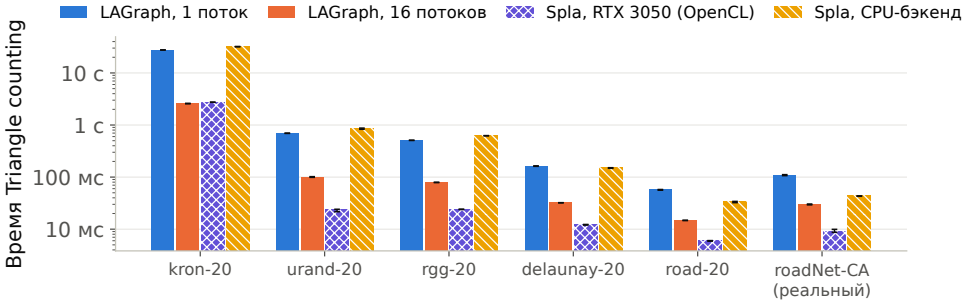
- Ошибка против точного PageRank 0.13–0.95 по ℓ_1 (у LAGraph $\sim 10^{-4}$).
- Встроенная проверка не ловит: эталон (строка 123) строится с той же ошибкой.
- Исправление: `A_degrees[Aj[k]]` в строках 87 и 123.

PageRank: Spla медленнее в 1.6–2.7 раза



- Исправленный пример (серые столбцы — как есть). Время итерации по 6 графам: Spla ≈ 4.7 мс $+ 0.53$ мс на млн рёбер, LAGraph ≈ 1.6 мс $+ 0.52$ мс: у Spla больше постоянная часть, а работа на ребро не меньше.
- **Постоянная часть втрое больше:** на итерацию 4 операции на GPU (`mxv`, два `eadd`, `reduce`) и чтение невязки на хост, чтобы решить, продолжать ли.
- **Работа на ребро не быстрее 16 потоков CPU:** ядро `mxv` отдаёт каждой строке матрицы варп из 32 потоков, а в строке в среднем 2.5–30 элементов (на road заняты 3 потока из 32); одновременно — не более 512 строк.

TC: Spla на GPU быстрее в 2.5–4 раза, кроме kron



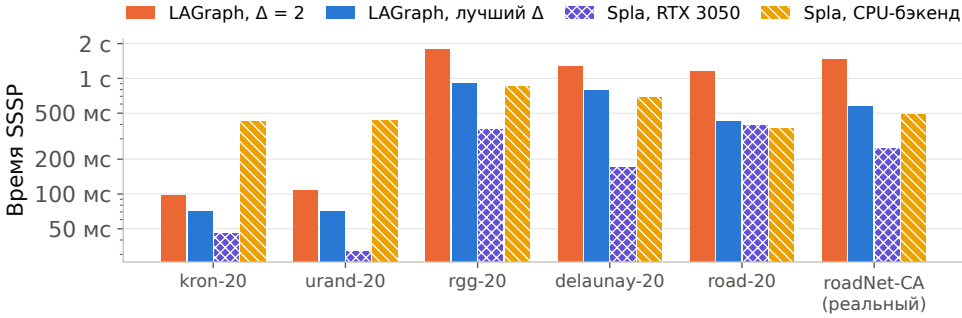
- TC — одно маскированное произведение $(L\,L^{\top}) \circ L$: шагов нет, накладные платятся один раз, а работа $\propto$ числу «клиньев». Тогда решает только скорость обработки клиньев: на urand и rgg GPU проходит их в 3–4 раза быстрее, чем LAGraph на 16 потоках.
- kron-20 — паритет (2.7 и 2.6 с): LAGraph видит перекос степеней (средняя > 3 медиан, `presort` = 1) и нумерует вершины по степени, Spla берёт L по исходным номерам.
- Посчитали работу на kron-20: пар рёбер через среднюю вершину по номерам $1.1 \cdot 10^{10}$, по степени $4.4 \cdot 10^{9}$ — в 2.5 раза меньше; на urand и rgg порядок не влияет. Это и съедает выигрыш GPU.

Выводы

1. **Где много работы на шаг — Spla на GPU быстрее**: SSSP в 2–5 раз, TC в 2.5–4 раза, BFS на широком фронте (urand) — в 2 раза.
2. **Где много шагов или мало работы на шаг — LAGraph быстрее**: у Spla уровень BFS не дешевле 0.17–0.3 мс, итерация PageRank — на 3 мс дороже (запуски ядер, синхронизация с хостом). BFS на дорогах медленнее в 1.6–4 раза, PageRank — в 1.6–2.7 раза.
3. **Эвристики важнее железа**: push/pull по числу вершин (kron, BFS) и порядок вершин в TC (kron) съедают выигрыш видеокарты; у LAGraph правило Бимера и сортировка по степени.
4. **Пример PageRank в Spla с ошибкой** (веса по строке): 1–2 итерации вместо 8–34; сравнивали исправленный.

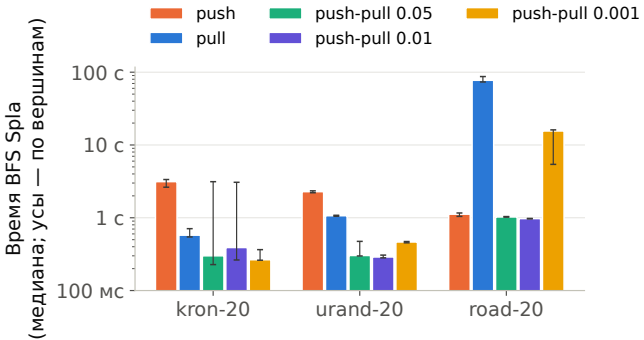
Код, данные и команды: `github.com/…/graphs-2026`. Повтор на той же машине в другой сессии — медианы в пределах 15 %, ни один вывод не меняется.

Приложение: параметр Δ в SSSP LAGraph



- Delta-stepping раскладывает вершины по «корзинам» шириной Δ: малое Δ — ближе к Дейкстре (много мелких шагов), $\Delta = \infty$ — Bellman–Ford, как у Spla. В основных замерах $\Delta = 2$ — значение из бенчмарка LAGraph.
- Перебор $\Delta = 1 \ldots \infty$ в одной сессии со Spla: лучший — ∞ (на kron — 16), LAGraph быстрее в 1.4–2.8 раза.
- И при лучшем Δ Spla на GPU быстрее в **1.6–4.7** раза; road-20 — паритет (0.39 против 0.42 с).

Приложение: порог push/pull в Spla (BFS, 2^{20})



- Spla переходит на pull, когда во фронте больше `front_factor` вершин (по умолчанию 5 %).
- kron: порог 0.001 убирает «дорогой» push по хабам (источник 177991: 3.1 → 0.26 с).
- urand: при том же пороге медленнее; road: pull на каждом из 1500 уровней — катастрофа.
- Один порог по числу вершин не подходит всем графам; LAGraph с правилом Бимера (по рёбрам) быстрее на всех трёх.